

Explanation – HW10: Representation Matrix & Change of Basis

Part 1: Foundations – explained simply

1. Coordinate vector: “How do I build v from the basis vectors?”

If I have a basis $B = \{b_1, b_2, \dots\}$, then every vector v can be written as a mixture of the basis vectors:

$$v = \alpha b_1 + \beta b_2 + \dots$$

The numbers $\alpha, \beta, \dots$ are the **coordinates of v with respect to B** . We write $[v]_B = (\alpha, \beta, \dots)$.

Analogy

A basis is like a **recipe kit**. The basis vectors are the ingredients. $[v]_B$ says *how much* of each ingredient I need to cook v .

In the standard basis this is trivial: $(3, 5)$ simply means “3 to the right, 5 up”. In another basis you first have to figure out how much of each (slanted) basis vector you need.

How to compute $[v]_B$

Set up $v = \alpha b_1 + \beta b_2 + \dots$, write it out componentwise, and solve the small system of equations for $\alpha, \beta, \dots$.

2. Representation matrix $[T]_B$: “What does T do, in basis language?”

Instead of describing T with formulas, you can represent T by a matrix – but the matrix depends on which basis you work in.

Construction of $[T]_B$

1. Apply T to each basis vector: $T(b_1), T(b_2), \dots$
2. Write each result **in basis- B -coordinates**: $[T(b_1)]_B, [T(b_2)]_B, \dots$
3. These coordinate vectors are the **columns** of $[T]_B$.

Analogy

$[T]_B$ is like a **translator handbook**: “If you give me a vector in B -coordinates, I’ll tell you where T sends it – again in B -coordinates.” The computation rule is then very simple: $[T(v)]_B = [T]_B \cdot [v]_B$.

3. Change of basis: translating between the “languages”

Often one basis is more convenient than another. The **change of basis** translates between them.

The change-of-basis matrix P

P has as its **columns the basis vectors of B , in standard coordinates**. Then:

$$[v]_E = P [v]_B \quad (\text{from } B\text{-language to standard})$$

$$[v]_B = P^{-1} [v]_E \quad (\text{from standard to } B\text{-language})$$

Change of basis for transformations

$$[T]_E = P [T]_B P^{-1} \quad \text{and conversely} \quad [T]_B = P^{-1} [T]_E P.$$

Memory aid: P builds standard out of B , P^{-1} undoes it. The matrix gets “converted from both sides”.

Analogy

Imagine two languages (standard and B). P is the dictionary “ $B \rightarrow$ standard”, P^{-1} is “standard $\rightarrow B$ ”. To understand a transformation in the other language, you translate: first into standard (P), apply the transformation there, then back (P^{-1}) – or exactly the other way around.

4. Two bases at once (Exercise 41)

If $T: V \rightarrow W$ connects different spaces, you have a basis B for the source and a basis C for the target. The matrix is called $[T]_B^C$.

Formula

$$[T]_B^C = P_C^{-1} A P_B,$$

where A is the matrix of T in the standard bases, P_B builds source-standard out of B , P_C builds target-standard out of C .

Part 2: Overview of the exercises

Exercise 39 – $[T]_B$ and $[T(v)]_B$ in two ways each

Here you see particularly nicely that both methods give the same result:

- **Method 1 (direct):** Compute the images of the basis vectors and rewrite them in B -coordinates. A bit more “converting”, but no P^{-1} needed.
- **Method 2 (change of basis):** $[T]_E$ is quickly written down, then convert with $P^{-1}[T]_E P$.

Conveniently: here $\det P = 1$, so P^{-1} is very easy.

Exercise 40 – from $[T]_B$ to the formula

The reverse direction: you know $[T]_B$ and want the normal formula $T(x, y)$. Trick: compute $[T]_E = P [T]_B P^{-1}$, then read off the formula directly from $[T]_E$ ($[T]_E$ applied to (x, y)).

Exercise 41 – two different bases

Like Exercise 39, but source and target have different bases. The logic stays: columns of $[T]_B^C$ are the images of the B -vectors, expressed in C -coordinates. In (b) you check the basic equation $[T]_B^C[v]_B = [T(v)]_C$ – it must hold for every v .

Exercise 42 – transformation on matrices

Trick

Even though T acts on 2×2 matrices, everything is as usual: the space $\mathbb{R}^{2 \times 2}$ is 4-dimensional, the standard basis consists of the four E_{ij} . A matrix $\begin{pmatrix} a & b \\ c & d \end{pmatrix}$ simply has the coordinates (a, b, c, d) .

Procedure:

- **(a)** Apply T to each of the four basis matrices, write the result as a 4-coordinate vector; these are the columns of $[T]_E$ (a 4×4 matrix).
- **(b)** With $[T]_E$ you compute kernel and image just like with normal vectors (Gauss, pivots, free variables) – at the end translate the coordinate vectors back into matrices.

Analogy

$T(X) = AX - XA$ measures “how far X is from commuting with A ”. The kernel contains exactly those matrices that commute with A ($AX = XA$). That is why A itself and the identity matrix I are in the kernel – they always commute with A .